

Время: 00:00–12:00

ЭКРАН: доска, блок ЧЗ → VS Code, `03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server`.

ГОВОРЮ

«В предыдущей части мы разобрали, как scheduler выбирает ноду. Теперь посмотрим, что происходит после назначения: какие ограничения получает процесс, почему один контейнер замедляется, а другой завершается, и кто добавляет Pod'ы или ноды.

DEVOPS MAY CRY остаётся тем же приложением. Мы не меняем бинарник под каждый вывод. Меняем policy вокруг него: requests, limits, QoS, HPA и VPA. Так видно, что поведение создаёт Kubernetes-конфигурация, а не скрытая разница между тестовыми приложениями.

Масштабирование начинается не с HPA, а с измерения. В обычном кластере `kubectl top` не читает данные прямо из kubelet. Команда идёт в aggregated API `metrics.k8s.io`, который обслуживает Metrics Server. Он хранит короткий актуальный срез CPU и memory. Это не замена Prometheus: истории, алертов и произвольных метрик здесь нет.

Сначала фиксирую корень клона, два kube-context и pinned Kubespray. До изменения проверяю, существует ли Metrics API. Ошибка `Metrics API not available` в этой точке — ожидаемое состояние, а не поломка стенда».

ПОКАЗАТЬ ДО КОМАНДЫ

- `03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml`.
- `metrics_server_enabled: true`.

- Учебный параметр `metrics_server_kubelet_insecure_tls: true`.
- `kubespary/VERSION` — upstream закреплён, не скачивается из случайной ветки.

📖 V2-C3-S01-C01 · Зафиксировать контекст и состояние Metrics API до установки

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT" || exit
set -a
source recording/.recording.env
set +a
KUBESPRAY_ROOT="$REPO_ROOT/kubespary"
KUBESPRAY_UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespary"
KUBESPRAY_INVENTORY="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
ANSIBLE_PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
test "$(git -C "$KUBESPRAY_UPSTREAM" describe --tags --exact-match)" = "v2.31.0"
kubectl --context "$SELF_CONTEXT" get nodes
kubectl --context "$SELF_CONTEXT" get apiservice v1beta1.metrics.k8s.io || true
kubectl --context "$SELF_CONTEXT" top nodes || true
sed -n '1,80p' 03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml
```

✅ После команды

Ожидая: три Ready-ноды self-managed кластера; до установки Metrics API либо отсутствует, либо `kubectl top` сообщает, что API недоступен. Pinned upstream — ровно `v2.31.0`.

🗣️ ГОВОРЮ ПЕРЕД УСТАНОВКОЙ

«Копирую два параметра в `group_vars` нашего `inventory` и запускаю официальный `cluster.yml` только с тегом `metrics_server`. Ansible-вызов и выбранная роль остаются видны в кадре.

После `PLAY RECAP` проверяю три разных слоя. Deployment должен раскатиться. Aggregated APIService должен стать Available. И только затем сырой API и `kubectl top` должны вернуть samples».

☰ V2-C3-S01-C02 · Установить Metrics Server ролью Kubespray и доказать API

```
install -m 0644 \  
    "$REPO_ROOT/03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml" \  
 \  
    "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/addons.yml" \  
pushd "$KUBESPRAY_UPSTREAM" >/dev/null \  
"$ANSIBLE_PLAYBOOK" -i "$KUBESPRAY_INVENTORY" \  
    cluster.yml \  
    --become --private-key "$SSH_KEY" --tags metrics_server \  
popd >/dev/null \  
kubectl --context "$SELF_CONTEXT" -n kube-system \  
    rollout status deploy/metrics-server --timeout=180s \  
kubectl --context "$SELF_CONTEXT" wait --for=condition=Available \  
    apiservice/v1beta1.metrics.k8s.io --timeout=180s \  
kubectl --context "$SELF_CONTEXT" get apiservice v1beta1.metrics.k8s.io \  
kubectl --context "$SELF_CONTEXT" get --raw /apis/metrics.k8s.io/v1beta1/nodes \  
    | jq -r '.items[] | [.metadata.name,.usage.cpu,.usage.memory] | @tsv' \  
kubectl --context "$SELF_CONTEXT" top nodes \  
kubectl --context "$SELF_CONTEXT" top pods -A --sort-by=memory | head -15
```

✅ После команды

Ожидая: `PLAY RECAP` без failed, Deployment доступен, APIService имеет `Available=True`, сырой Metrics API и `kubectl top` возвращают samples.

ФИЗИЧЕСКИЙ СМЫСЛ

Metrics Server опрашивает kubelet summary API, а API Server публикует его через aggregation layer. HPA позже читает тот же resource metrics API. Сам `kubectl top` ничего не измеряет — он форматирует готовый ответ.

PRODUCTION NUANCE

В учебном стенде включён TLS-компромисс для self-signed kubelet serving certificates. В рабочей среде такой флаг не копируют автоматически: выпускают проверяемые serving-сертификаты, настраивают trust chain и проверяют APIService. Зелёный `kubectl top` также не подтверждает, что requests/limits выбраны правильно.

ПЕРЕХОД

«Метрики появились. Теперь откроем один Pod одновременно глазами scheduler и глазами Linux kernel».

с3.1 Requests для scheduler, limits для cgroup

Время: 12:00–25:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/22_3.1_requests_limits_cgroup/requests_limits.yaml` → терминал → SSH на worker.

ГОВОРЮ

«В `requests_limits.yaml` три Deployment используют один immutable image DEVOPS MAY CRY. Сейчас беру вариант `Burstable`: request `100m CPU` и `64Mi memory`, limit `500m` и `256Mi`.

Request отвечает на вопрос scheduler: на какой ноде зарезервировать место. Limit отвечает на другой вопрос: сколько процессу разрешит runtime. Scheduler не ждёт фактическую нагрузку и не читает cgroup. Он складывает requests из Pod spec.

Сначала применяю манифест, нахожу конкретный Pod и ноду. Затем показываю его QoS и секцию `Allocated resources` этой ноды».

ПОКАЗАТЬ

- Один и тот же digest image у всех трёх Deployment.
- У `devops-may-cry-burstable`: requests меньше limits.
- `automountServiceAccountToken: false`, non-root, dropped capabilities, read-only root filesystem.
- Не листать весь YAML: остановиться на `resources` и `securityContext`.

V2-C3-S02-C01 · Применить requests/limits и увидеть решение планировщика

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/22_3.1_requests_limits_cgroup"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" create namespace traffic-lab \
  --dry-run=client -o yaml | kubectl --context "$SELF_CONTEXT" apply -f -
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  rollout status deploy/devops-may-cry-burstable --timeout=180s
POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry-burstable -o jsonpath='{.items[0].metadata.name}')"
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o jsonpath='{.spec.nodeName}')"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o custom-
columns='NAME:.metadata.name,NODE:.spec.nodeName,QOS:.status.qosClass,REQUEST_CPU:
.spec.containers[0].resources.requests.cpu,LIMIT_CPU:.spec.containers[0].resources.limits.cpu,REQUEST_MEM:.spec.containers[0].resources.requests.memory,LIMIT_MEM:
.spec.containers[0].resources.limits.memory'
kubectl --context "$SELF_CONTEXT" describe node "$NODE" \
  | sed -n '/Allocated resources:/,/Events:/p'
```

После команды

Ожидая: Pod `Burstable`, request `100m/64Mi`, limit `500m/256Mi`; в `Allocated resources` ноды requests и limits считаются отдельно.

ГОВОРЮ ПЕРЕД SSH

«Теперь проверю, во что kubelet и container runtime превратили limit. В контейнере нет shell и `cat`: образ distroless. Это не недостаток приложения, а уменьшение поверхности атаки.

Поэтому беру container ID из Pod status, а `nodeName` автоматически сопоставляю с `NODE1_SSH_HOST`, `NODE2_SSH_HOST` или `NODE3_SSH_HOST` из локального env. Адрес руками не угадываю. Через SSH и `crictl inspect` получаю PID и читаю cgroup из mount namespace процесса. Public IP нужен только для SSH. Сам Pod работает в private cluster network».

🔧 V2-C3-S02-C02 · Прочитать реальные cgroup-ограничения через runtime

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab exec "$POD" \
  -- cat /sys/fs/cgroup/memory.max || true
CONTAINER_ID="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o jsonpath='{.status.containerStatuses[0].containerID}' | sed
's#^[^:]*://##')"
```

```
printf 'pod=%s node=%s container=%s\n' "$POD" "$NODE" "$CONTAINER_ID"
case "$NODE" in
  node1) NODE_SSH_HOST="{NODE1_SSH_HOST:?fill NODE1_SSH_HOST}" ;;
  node2) NODE_SSH_HOST="{NODE2_SSH_HOST:?fill NODE2_SSH_HOST}" ;;
  node3) NODE_SSH_HOST="{NODE3_SSH_HOST:?fill NODE3_SSH_HOST}" ;;
  *) printf 'unexpected node: %s\n' "$NODE" >&2; exit 1 ;;
esac
SSH_USER="{NODE02_SSH_USER:-root}"
REMOTE_PID="$(ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  "sudo crictl inspect '$CONTAINER_ID'" \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["info"]["pid"])')"
```

```
ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  "sudo nsenter -t '$REMOTE_PID' -m cat /sys/fs/cgroup/memory.max; sudo nsenter -
t '$REMOTE_PID' -m cat /sys/fs/cgroup/cpu.max"
```

✅ После команды

Ожидая: `kubectl exec` не находит `cat` в distroless-образе; через runtime видны конечный `memory.max` для 256Mi и CPU quota, соответствующая limit `500m`.

ФИЗИЧЕСКИЙ СМЫСЛ

Для cgroup v2 `memory.max` — предел памяти в байтах. `cpu.max` содержит quota и period: при `500m` контейнер получает примерно половину одного CPU во времени. Kubernetes Quantity из YAML превратился в параметры Linux kernel.

ЛОВУШКА

Не устанавливать shell в production image ради диагностики. Для редких случаев есть ephemeral debug container, runtime-инструменты на ноде и отдельные debug-образы. Доступ к ноде должен быть ограничен и аудироваться.

ПЕРЕХОД

«Мы записали память с суффиксом `Mi`. Сейчас покажу, почему это не косметика».

Время: 25:00–32:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/23_3.2_memory_units/oom_cpu_demo.yaml`, затем терминал.

ГОВОРЮ

«Kubernetes использует формат Quantity. Для CPU `1` означает одно ядро, `1000m` — то же самое, `100m` — десятую часть. Для памяти bare integer — байты. Поэтому `memory: 128` означает не 128 мегабайт, а 128 байт.

`M` — десятичные мегабайты, `Mi` — двоичные mebibytes. Обычно для памяти в манифестах пишут `Mi` и `Gi`, чтобы не гадать о единицах.

В `oom_cpu_demo.yaml` нормальный limit записан как `96Mi`. Для контраста создаю один временный Pod того же DEVOPS MAY CRY image с limit `128`. Полный security context передаю в объекте, поэтому Pod проходит restricted policy namespace. Жду не абстрактный CrashLoop, а конкретный `OOMKilled` и exit code».

ПОКАЗАТЬ

- В `oom_cpu_demo.yaml`: `requests.memory: 64Mi`, `limits.memory: 96Mi`.
- Команду с ошибочным `"memory": "128"`.
- После запуска — jsonpath с исходным limit, reason и exit code.

■ V2-C3-S03-C01 · Доказать, что memory 128 означает 128 байт

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod bad-memory-unit \
  --ignore-not-found --wait=true
kubectl --context "$SELF_CONTEXT" -n traffic-lab run bad-memory-unit \
  --image='ghcr.io/boost-mentor/devops-may-
cry@sha256:41439418d570649055117277a53b17b91fb40dad33dc3fa6b8a151fe68f86ca7' \
  --restart=Always \
  --overrides='{"spec":{"automountServiceAccountToken":false,"securityContext":
{"runAsNonRoot":true,"runAsUser":65532,"runAsGroup":65532,"seccompProfile":
{"type":"RuntimeDefault"}},"containers":[{"name":"bad-memory-
unit","image":"ghcr.io/boost-mentor/devops-may-
cry@sha256:41439418d570649055117277a53b17b91fb40dad33dc3fa6b8a151fe68f86ca7"},"res
ources":{"limits":{"memory":"128"}},"securityContext":
{"allowPrivilegeEscalation":false,"capabilities":{"drop":
["ALL"]},"readOnlyRootFilesystem":true,"runAsNonRoot":true,"runAsUser":65532,"run
AsGroup":65532}}]}'
for _attempt in $(seq 1 30); do
  REASON="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod bad-memory-
unit \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'
2>/dev/null || true)"
  test "$REASON" = "OOMKilled" && break
  sleep 2
done
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod bad-memory-unit \
  -o jsonpath='limit={.spec.containers[0].resources.limits.memory}{" reason="}
{.status.containerStatuses[0].lastState.terminated.reason}{" exit="}
{.status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}'
test "$REASON" = "OOMKilled"
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod bad-memory-unit --
wait=true
```

✅ После команды

Ожидая: в объекте записан limit `128`, процесс завершается `OOMKilled` с exit code `137`; после доказательства Pod удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

API Server не исправляет смысл значения за автора. Он принимает валидную Quantity, runtime создаёт почти нулевой cgroup limit, а kernel завершает процесс, который физически не может уместиться.

⚠️ PRODUCTION NUANCE

Такие ошибки лучше ловить до кластера: schema validation, policy engine и review правил на минимальные/максимальные ресурсы. Одной проверки, что YAML парсится, недостаточно.

ПЕРЕХОД

«Даже с правильными единицами нельзя делить всю память ноды между приложениями. Часть ресурсов Kubernetes оставляет самой машине».

с3.3 Capacity и Allocatable — не одно число

Время: 32:00–39:00

ЭКРАН: терминал, объект `Node/node2`; локального манифеста в этой сцене нет.

ГОВОРЮ

«Capacity — то, что kubelet увидел на машине. Allocatable — то, что нода объявляет доступным Pod'ам после системных резервов и eviction threshold. Scheduler планирует по Allocatable.

В этой лаборатории нет YAML, который создаёт Capacity: это status объекта Node, его заполняет kubelet. Сравниваю оба поля на `node2`, затем смотрю уже назначенные requests и limits».

V2-C3-S04-C01 · Сравнить Capacity и Allocatable на worker-ноде

```
kubectl --context "$SELF_CONTEXT" describe node node2 \  
  | sed -n '/Capacity:\/,\/System Info:/p'  
kubectl --context "$SELF_CONTEXT" get node node2 \  
  -o jsonpath='capacity={.status.capacity}{"\n"}allocatable={.status.allocatable}{"\n"}'  
kubectl --context "$SELF_CONTEXT" describe node node2 \  
  | sed -n '/Allocated resources:\/,\/Events:/p'
```

После команды

Ожидая: Node публикует оба набора значений; Allocatable отражает доступное Pod'ам после резервов и порогов kubelet, а `Allocated resources` показывает сумму requests/limits уже назначенных Pod'ов.

ФИЗИЧЕСКИЙ СМЫСЛ

Упрощённо: $\text{Allocatable} = \text{Capacity} - \text{system reserved} - \text{kube reserved} - \text{eviction reserve}$. Точная разница зависит от конфигурации kubelet и ОС. Если reservations не настроены, системные процессы всё равно потребляют ресурсы, но scheduler может этого не учесть достаточно консервативно.

ЛОВУШКА

Нельзя обещать приложениям 100% RAM ноды. Kernel, runtime, kubelet и системные DaemonSet тоже должны пережить пик. Для capacity planning смотрят Allocatable, DaemonSet overhead и реальное использование, а не только паспорт VM.

ПЕРЕХОД

«Requests должны помещаться в Allocatable. А вот limits Kubernetes разрешает продать несколько раз».

Время: 39:00–49:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/25_3.4_overcommit/requests_limits.yaml` → терминал.

ГОВОРЮ

«У Burstable-варианта request CPU `100m`, limit `500m`. Масштабирую его до двенадцати реплик на двух workers.

Scheduler проверяет сумму requests. Поэтому Pod'ы могут разместиться, даже если сумма limits на ноде превышает 100%. Это overcommit: мы рассчитываем, что не все контейнеры одновременно потребуют максимум.

После rollout показываю `Allocated resources` на node2 и node3. Затем удаляю только три Deployment этой лабы, чтобы следующий отказ не смешивался с фоновыми Pod'ами».

V2-C3-S05-C01 · Показать overcommit без искусственного отказа ноды

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/25_3.4_overcommit"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry-burstable --replicas=12
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry-burstable --timeout=180s
for node in node2 node3; do
    printf '\n=== %s ===\n' "$node"
    kubectl --context "$SELF_CONTEXT" describe node "$node" \
        | sed -n '/Allocated resources:/,/Events:/p'
done
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
    -l app=devops-may-cry-burstable -o wide
kubectl --context "$SELF_CONTEXT" delete -f requests_limits.yaml --ignore-not-found
```

После команды

Ожидаю: все 12 Pod'ов планируются, потому что помещается сумма requests; сумма CPU limits на worker-нодах может быть выше 100%. В конце три лабораторных Deployment удалены.

ФИЗИЧЕСКИЙ СМЫСЛ

CPU — compressible resource: при конкуренции процессы получают меньше CPU и работают дольше. Memory — non-compressible: если физическая память закончилась, кто-то будет reclaim'иться, evict'иться или получит OOM. Поэтому одинаковый процент overcommit для CPU и memory имеет разный риск.

⚠ **PRODUCTION NUANCE**

Overcommit выбирают по профилю нагрузки, p95/p99, SLO и запасу ноды. Если все ночные batch-задачи стартуют одновременно, предположение «пики не совпадут» перестаёт работать. Scarcity policy должна учитывать такой сценарий.

ПЕРЕХОД

«Теперь специально превысим memory limit одного контейнера и отделим cgroup OOM от нехватки памяти на всей ноде».

с3.5 OOMKilled: limit одного контейнера

Время: 49:00–61:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/26_3.5_oomkilled/oom_cpu_demo.yaml` → терминал → коротко SSH.

ГОВОРЮ

«В первом документе `oom_cpu_demo.yaml` DEVOPS MAY CRY имеет request `64Mi` и limit `96Mi`. Endpoint `/wound` включён только переменной `LAB_ENDPOINTS_ENABLED=true` в этой изолированной лабе. В обычном base он недоступен.

Приложение выделяет 160Mi и касается каждой страницы памяти. Это важно: просто создать большой slice недостаточно, пока kernel физически не закоммитил страницы.

Применяю файл, оставляю только OOM-вариант, фиксирую restartCount и вызываю `/wound` одноразовым контейнером с тем же image. Жду, пока restartCount действительно увеличится».

ПОКАЗАТЬ

- `LAB_ENDPOINTS_ENABLED: "true"`.
- `limits.memory: 96Mi`.
- Readiness probe и restricted security context.
- Второй Deployment в файле существует для следующей сцены; сейчас он удаляется.

📄 V2-C3-S06-C01 · Воспроизвести cgroup OOM на DEVOPS MAY CRY

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/26_3.5_oomkilled"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f oom_cpu_demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete deploy devops-may-cry-throttled --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  rollout status deploy/devops-may-cry-oom --timeout=180s
OOM_POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry-oom -o jsonpath='{.items[0].metadata.name}')"
OOM_IP="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
  -o jsonpath='{.status.podIP}')"
IMAGE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-
cry-oom \
  -o jsonpath='{.spec.template.spec.containers[0].image}')"
RESTARTS_BEFORE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod
"$OOM_POD" \
  -o jsonpath='{.status.containerStatuses[0].restartCount}')"
kubectl --context "$SELF_CONTEXT" -n traffic-lab run wound-trigger \
  --rm -i --restart=Never --image="$IMAGE" \
  --env="HEALTHCHECK_URL=http://$OOM_IP:8080/wound?mb=160" \
  --command -- /devops-may-cry -healthcheck || true
for _attempt in $(seq 1 30); do
  RESTARTS_AFTER="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod
"$OOM_POD" \
    -o jsonpath='{.status.containerStatuses[0].restartCount}')"
  test "$RESTARTS_AFTER" -gt "$RESTARTS_BEFORE" && break
  sleep 2
done
test "$RESTARTS_AFTER" -gt "$RESTARTS_BEFORE"
```

✅ После команды

Ожидая: запрос к лабораторному `/wound` превышает limit `96Mi`, соединение может оборваться, а restartCount основного Pod'а увеличивается.

🗣 ГОВОРЮ ПЕРЕД ДОКАЗАТЕЛЬСТВОМ

«Оборванный HTTP-запрос ещё ничего не доказывает. Читаю `Last State` контейнера: reason должен быть `OOMKilled`, exit code — `137`. Это `128 + 9`, где 9 — `SIGKILL`.

Дополнительно смотрю kernel log на той ноде, которую вернул Pod spec; её SSH-адрес команда берёт из локального env. Эта строка зависит от ОС и политики доступа, поэтому переносимый acceptance criterion — Pod status. После проверки удаляю OOM workload».

📄 V2-C3-S06-C02 · Доказать OOMKilled в Pod status и kernel log

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab describe pod "$OOM_POD" \
  | sed -n '/Last State:\/,/Ready:/p'
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}{"
exit="}{.status.containerStatuses[0].lastState.terminated.exitCode}{" restarts="}
{.status.containerStatuses[0].restartCount}{"\n"}'
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
  -o jsonpath='{.spec.nodeName}')"
case "$NODE" in
  node1) NODE_SSH_HOST="${NODE1_SSH_HOST:?fill NODE1_SSH_HOST}" ;;
  node2) NODE_SSH_HOST="${NODE2_SSH_HOST:?fill NODE2_SSH_HOST}" ;;
  node3) NODE_SSH_HOST="${NODE3_SSH_HOST:?fill NODE3_SSH_HOST}" ;;
  *) printf 'unexpected node: %s\n' "$NODE" >&2; exit 1 ;;
esac
SSH_USER="${VIDE02_SSH_USER:-root}"
ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  'sudo dmesg -T | grep -iE "memory cgroup out of memory|killed process" | tail
-5' || true
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete deploy devops-may-cry-oom --ignore-not-found
```

✅ После команды

Ожидая: `Last State` содержит `Reason: OOMKilled` и exit code `137`; kernel log может добавить строку cgroup OOM, но главным переносимым доказательством остаётся Pod status. Workload удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Здесь нода могла иметь свободную память. Процесс завершил memory controller именно его cgroup, потому что контейнер превысил собственный limit. kubelet затем перезапустил контейнер внутри того же Pod по restart policy Deployment.

⚠ ЛОВУШКА

Exit `137` часто указывает на SIGKILL, но сам по себе не называет источник. Сверяем `lastState.terminated.reason`, события, метрики и kernel log. `kubectl logs` текущего контейнера без `--previous` может быть пустым или показывать уже новый процесс.

ПЕРЕХОД

«С памятью процесс завершился. На CPU limit результат другой: процесс останется жив, но будет ждать quota».

Время: 61:00–72:00

ЭКРАН: второй документ

`03_ЧАСТЬ_3_SCALING/27_3.6_cpu_throttling/oom_cpu_demo.yaml` → терминал.

ГОВОРЮ

«У throttled-варианта request `100m`, limit `200m`. Endpoint `/overload` запускает две CPU-bound goroutine на 120 секунд. Контейнер попытается взять больше quota, чем разрешено.

До нагрузки читаю cAdvisor counter `container_cpu_cfs_throttled_periods_total`. После запроса жду двадцать секунд и читаю тот же counter. Проверяю два условия: значение выросло, restartCount остался нулём».

ПОКАЗАТЬ

- `limits.cpu: 200m`.
- Тот же digest DEVOPS MAY CRY.
- Команду выборки одного Pod из cAdvisor metrics по container и pod label.

■ V2-C3-S07-C01 · Доказать CPU throttling счётчиком, а не перезапуском

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/27_3.6_cpu_throttling"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f oom_cpu_demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    delete deploy devops-may-cry-oom --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry-throttled --timeout=180s
CPU_POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
    -l app=devops-may-cry-throttled -o jsonpath='{.items[0].metadata.name}')"
CPU_IP="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
    -o jsonpath='{.status.podIP}')"
IMAGE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-
cry-throttled \
    -o jsonpath='{.spec.template.spec.containers[0].image}')"
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
    -o jsonpath='{.spec.nodeName}')"
BEFORE="$(kubectl --context "$SELF_CONTEXT" get --raw
"/api/v1/nodes/$NODE/proxy/metrics/cadvisor" \
    | grep 'container_cpu_cfs_throttled_periods_total' \
    | grep 'container="devops-may-cry"' | grep "pod=\"$CPU_POD\"" \
    | awk '{print $2}' | head -1)"
test -n "$BEFORE"
kubectl --context "$SELF_CONTEXT" -n traffic-lab run overload-trigger \
    --rm -i --restart=Never --image="$IMAGE" \
    --env="HEALTHCHECK_URL=http://$CPU_IP:8080/overload?sec=120" \
    --command -- /devops-may-cry -healthcheck
sleep 20
kubectl --context "$SELF_CONTEXT" -n traffic-lab top pod "$CPU_POD"
AFTER="$(kubectl --context "$SELF_CONTEXT" get --raw
"/api/v1/nodes/$NODE/proxy/metrics/cadvisor" \
    | grep 'container_cpu_cfs_throttled_periods_total' \
    | grep 'container="devops-may-cry"' | grep "pod=\"$CPU_POD\"" \
```

```
| awk '{print $2}' | head -1)"
RESTARTS="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
-o jsonpath='{.status.containerStatuses[0].restartCount}')"
printf 'throttled periods: before=%s after=%s; restarts=%s\n' \
"$BEFORE" "$AFTER" "$RESTARTS"
awk -v before="$BEFORE" -v after="$AFTER" 'BEGIN { exit !(after > before) }'
test "$RESTARTS" -eq 0
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
delete deploy devops-may-cry-throttled --ignore-not-found
```

✅ После команды

Ожидая: throttled-period counter увеличивается, Pod остаётся Running, restartCount равен нулю. CPU limit замедлил процесс, но не убил его.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Linux CFS выдаёт процессу quota на period. Когда quota исчерпана, runnable thread ждёт следующего периода. Контейнер остаётся Running, но latency растёт. Поэтому «Pod не перезапускается» не означает «CPU limit не влияет».

⚠️ PRODUCTION NUANCE

Искать throttling только через `kubectl top` неудобно: top показывает usage sample, а не время ожидания quota. Для расследования нужны throttling counters рядом с latency, saturation и requests. Иногда CPU limit снимают, но это решение принимают по профилю нагрузки, а не как универсальное правило.

ПЕРЕХОД

«Requests и limits также определяют QoS-класс. Проверим классы на одном image, а потом разберём, чего QoS не гарантирует».

Время: 72:00–82:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/28_3.7_qos_oom/requests_limits.yaml` → терминал.

ГОВОРЮ

«В файле три Deployment одного приложения.

`Guaranteed`: у каждого контейнера заданы CPU и memory, request равен limit.

`BestEffort`: requests и limits нет вообще. Всё остальное попадает в `Burstable`.

Применяю файл и проверяю каждый класс как acceptance criterion. Не делаю вывод по имени Deployment — читаю `.status.qosClass`, который присвоил kubelet».

V2-C3-S08-C01 · Проверить три QoS-класса на одном image

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/28_3.7_qos_oom"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab wait --for=condition=Available \
  deploy/devops-may-cry-guaranteed \
  deploy/devops-may-cry-burstable \
  deploy/devops-may-cry-besteffort --timeout=180s
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
  -l lab.boostmentor.dev/qos \
  -o custom-
columns='NAME:.metadata.name,QOS:.status.qosClass,NODE:.spec.nodeName'
for expected in \
  guaranteed:Guaranteed \
  burstable:Burstable \
  best-effort:BestEffort; do
  label="${expected%*:}"
  wanted="${expected###:}"
  pod="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
    -l "lab.boostmentor.dev/qos=$label" \
    -o jsonpath='{.items[0].metadata.name}')"
  actual="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$pod" \
    -o jsonpath='{.status.qosClass}')"
  printf '%-12s expected=%-10s actual=%s\n' "$label" "$wanted" "$actual"
  test "$actual" = "$wanted"
done
kubectl --context "$SELF_CONTEXT" delete -f requests_limits.yaml --ignore-not-found
```

✅ После команды

Ожидая: один и тот же immutable image получает классы `Guaranteed`, `Burstable`, `BestEffort` только из-за разных resources; все три проверки проходят, затем workload удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

QoS влияет на параметры cgroup и на поведение kubelet при pressure. Но фраза «BestEffort всегда умирает первым» слишком грубая. При memory pressure kubelet учитывает, превышает ли Pod request, его Priority и насколько usage превышает request. При cgroup OOM внутри собственного limit картина снова другая.

⚠️ ЛОВУШКА

Guaranteed не означает «никогда не завершится» и не создаёт дополнительную память. Он фиксирует reservation и limit. Для критичного workload дополнительно нужны PriorityClass, topology, PDB, capacity и нормальные probes.

ПЕРЕХОД

«OOMKilled был решением memory controller для процесса. Eviction — решение kubelet для Pod. Получим второй результат отдельно».

с3.8 Evicted — это не OOMKilled

Время: 82:00–92:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/29_3.8_eviction/eviction-demo.yaml` → терминал.

ГОВОРЮ

«Я не буду забивать память или диск всей worker-ноды. Лаба ограничена одним Pod.

В `eviction-demo.yaml` контейнер пишет 64Mi в `emptyDir`. У volume и container ephemeral-storage limit — 16Mi. `emptyDir.sizeLimit` не создаёт отдельный маленький диск; kubelet периодически считает local ephemeral storage и, когда Pod превышает лимит, evict'ит его.

До запуска показываю, что у нод нет MemoryPressure, DiskPressure и PIDPressure.

Затем применяю один Pod и жду конкретный `.status.reason=Evicted`».

ПОКАЗАТЬ

- `restartPolicy: Never`.
- `requests.ephemeral-storage: 8Mi`, `limits: 16Mi`.
- `emptyDir.sizeLimit: 16Mi`.
- Restricted security context и pinned debug image.

📄 V2-C3-S09-C01 · Получить Evicted без давления на всю ноду

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/29_3.8_eviction"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" get nodes \
  -o custom-columns='NAME:.metadata.name,MEMORY:.status.conditions[?
(@.type=="MemoryPressure")].status,DISK:.status.conditions[?
(@.type=="DiskPressure")].status,PID:.status.conditions[?
(@.type=="PIDPressure")].status'
sed -n '1,220p' eviction-demo.yaml
kubectl --context "$SELF_CONTEXT" apply -f eviction-demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab wait \
  --for=jsonpath='{.status.reason}'=Evicted \
  pod/ephemeral-storage-hog --timeout=300s
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod ephemeral-storage-hog \
  -o jsonpath='phase={.status.phase}{" reason="}{.status.reason}{"\nmessage="}
{.status.message}{"\n"}'
test "$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod ephemeral-
storage-hog \
  -o jsonpath='{.status.reason}')" = "Evicted"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get events \
  --field-selector involvedObject.name=ephemeral-storage-hog \
  --sort-by=.lastTimestamp
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete pod ephemeral-storage-hog --ignore-not-found
```

✅ После команды

Ожидая: отдельный Pod получает phase `Failed`, reason `Evicted` из-за превышения своего ephemeral-storage limit; NodePressure не создаётся намеренно. Pod удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Eviction переводит Pod в phase `Failed` и reason `Evicted`. Это объект уровня kubelet/Pod, не lastState `OOMKilled` одного процесса. Поскольку здесь standalone Pod, controller не создаёт замену. У Deployment на его месте ReplicaSet создал бы новый Pod.

⚠️ PRODUCTION NUANCE

Причиной eviction может быть node pressure или Pod-level превышение local storage. Поэтому рядом с reason читают message, events и Node conditions. Массовое удаление всех Failed Pod'ов стирает доказательства и затрагивает чужие workload — здесь удаляем только свой Pod.

ПЕРЕХОД

«Теперь у нас есть метрики и корректные requests. Этого достаточно, чтобы HPA посчитал нужное число реплик».

с3.9 HPA: больше Pod'ов по CPU utilization

Время: 92:00–106:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/30_3.9_hpa/app` → `hpa_lab.yaml` → терминал.

ГОВОРЮ

«Base DEVOPS MAY CRY выглядит как обычный workload: Deployment, ClusterIP Service, PDB, probes, requests/limits и restricted security context. Endpoint нагрузки в base выключен. Overlay `recording` добавляет только `LAB_ENDPOINTS_ENABLED=true` для этой сцены.

В `hpa_lab.yaml` target — Deployment, минимум две реплики, максимум шесть, CPU utilization 50%. Utilization считается относительно CPU request контейнера. Без request HPA по этой метрике не сможет посчитать процент.

Сначала применяю overlay, отдельный debug client и HPA. Проверяю baseline: две Ready-реплики, request `100m`, HPA видит метрику».

ПОКАЗАТЬ

- `app/base/deployment.yaml`: resources и probes.
- `app/overlays/recording/enable-lab-endpoints.yaml`: одна env-переменная.
- `hpa_lab.yaml`: min/max, target 50%, scaleUp и scaleDown behavior.
- `debug-client.yaml`: отдельный ограниченный Pod, а не shell внутри production image.

📄 V2-C3-S10-C01 · Подготовить HPA с Metrics API и контролируемым endpoint

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/30_3.9_hpa"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -k app/overlays/recording
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$SELF_CONTEXT" apply -f debug-client.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  wait --for=condition=Ready pod/client --timeout=180s
kubectl --context "$SELF_CONTEXT" apply -f hpa_lab.yaml
CURRENT_UTILIZATION=""
for _attempt in $(seq 1 18); do
  CURRENT_UTILIZATION="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    get hpa devops-may-cry \
    -o jsonpath='{.status.currentMetrics[0].resource.current.averageUtilization}'
  \
    2>/dev/null || true)"
  test -n "$CURRENT_UTILIZATION" && break
  sleep 5
done
test -n "$CURRENT_UTILIZATION"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-cry
kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-cry \
  -o custom-
columns='NAME:.metadata.name,REPLICAS:.status.readyReplicas,REQUEST_CPU:.spec.template.spec.containers[0].resources.requests.cpu'
```

✅ После команды

Ожидая: Deployment имеет две Ready-реплики и CPU request `100m`; HPA видит текущую метрику, target `50%`, min `2`, max `6`.

ГОВОРЮ ПЕРЕД НАГРУЗКОЙ

«Из debug client отправляю восемь запросов к `/overload`. Endpoint быстро отвечает, а CPU-работа продолжается девяносто секунд внутри Pod'ов.

Упрощённая формула HPA: $\text{ceil}(\text{current replicas} \times \text{current utilization} / \text{target utilization})$. Контроллер применяет tolerance и behavior policy, поэтому число меняется не мгновенно и не обязано перескочить сразу на максимум.

Жду, пока desiredReplicas станет больше двух, показываю новые Pod'ы и затем возвращаю стенд к baseline».

V2-C3-S10-C02 · Создать CPU-нагрузку и доказать горизонтальное масштабирование

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab exec client -- sh -c \
  'for i in $(seq 1 8); do curl -fsS "http://devops-may-cry/overload?sec=90"
  >/dev/null & done; wait'
for _attempt in $(seq 1 24); do
  kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-cry
  DESIRED="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-
  cry \
    -o jsonpath='{.status.desiredReplicas}')"
  test "${DESIRED:-0}" -gt 2 && break
  sleep 5
done
test "$DESIRED" -gt 2
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
  -l app=devops-may-cry -o wide
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete hpa devops-may-cry
kubectl --context "$SELF_CONTEXT" -n traffic-lab scale \
  deploy/devops-may-cry --replicas=2
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod client --wait=true
```

✅ После команды

Ожидая: CPU utilization превышает target, desiredReplicas становится больше двух, появляются дополнительные Pod'ы, но не больше шести. После доказательства HPA и клиент удалены, Deployment возвращён к двум репликам.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

HPA меняет поле replicas у target workload. Он создаёт Pod'ы через Deployment/ReplicaSet, но не создаёт VM. Если новым Pod'ам некуда сесть, они останутся Pending — это уже задача Cluster Autoscaler.

⚠️ ЛОВУШКА

Один короткий spike, отсутствующий request или запаздывающий Metrics API дают другой результат. Scale-down обычно стабилизируют дольше, чтобы реплики не дёргались туда-сюда. В рабочей среде target выбирают по latency, saturation и стоимости, а не потому что 50% выглядит аккуратно.

ПЕРЕХОД

«HPA меняет количество Pod'ов. VPA отвечает на другой вопрос: какой request разумно дать одному Pod'у».

Время: 106:00–120:00 плюс ожидание samples

ЭКРАН: `03_ЧАСТЬ_3_SCALING/31_3.10_vpa/vpa_lab.yaml` → pinned upstream manifests → терминал.

ГОВОРЮ

«VPA состоит из CRD и контроллеров. Для этой сцены нужны CRD, RBAC и recommender. Updater и admission-controller не ставлю: я не хочу, чтобы лаборатория сама пересоздала workload или переписала request.

Source закреплён одновременно тегом `vertical-pod-autoscaler-1.7.0` и точным commit. Перед apply показываю upstream `recommender-deployment.yaml`, затем применяю конкретные CRD/RBAC/recommender manifests. Каждый устанавливаемый объект остаётся виден».

ПОКАЗАТЬ

- `vpa_lab.yaml`: targetRef на DEVOPS MAY CRY.
- `updateMode: "Off"`.
- minAllowed/maxAllowed и controlledResources.
- В терминале — exact tag/commit и отсутствие updater/admission-controller.

📄 V2-C3-S11-C01 · Установить только VPA recommender из pinned source

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/31_3.10_vpa"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -k base
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    delete hpa devops-may-cry --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry --timeout=180s
VPA_VERSION="1.7.0"
VPA_COMMIT="6a616ea0c5ea0cb6111240073a9273b3467c064e"
VPA_SOURCE="${XDG_CACHE_HOME:-$HOME/.cache}/video2-vpa/autoscaler"
if [[ ! -d "$VPA_SOURCE/.git" ]]; then
    mkdir -p "$(dirname "$VPA_SOURCE")"
    git clone --depth 1 --branch "vertical-pod-autoscaler-$VPA_VERSION" \
        https://github.com/kubernetes/autoscaler.git "$VPA_SOURCE"
fi
test "$(git -C "$VPA_SOURCE" rev-parse HEAD)" = "$VPA_COMMIT"
VPA_DEPLOY="$VPA_SOURCE/vertical-pod-autoscaler/deploy"
test -z "$(kubectl --context "$SELF_CONTEXT" -n kube-system get deploy \
    vpa-updater vpa-admission-controller --ignore-not-found -o name)"
sed -n '1,80p' "$VPA_DEPLOY/recommender-deployment.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/vpa-v1-crd-gen.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/vpa-rbac.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/recommender-
deployment.yaml"
kubectl --context "$SELF_CONTEXT" -n kube-system \
    rollout status deploy/vpa-recommender --timeout=180s
kubectl --context "$SELF_CONTEXT" get crd \
    verticalpodautoscalers.autoscaling.k8s.io
```

✅ После команды

Ожидаю: source имеет exact commit, установлены CRD/RBAC/recommender; updater и admission-controller отсутствуют, recommender доступен.

🗣️ ГОВОРЮ ПЕРЕД ОЖИДАНИЕМ

«Перед созданием VPA сохраняю UID текущих Pod'ов. Recommender нужна история metrics, поэтому опрашиваю не просто существование объекта, а непустую target recommendation.

Когда target CPU/memory появился, сравниваю UID. В `off` режиме они должны совпасть: VPA посчитал рекомендацию, но ничего не применил».

📖 V2-C3-S11-C02 · Получить рекомендацию VPA без пересоздания Pod'ов

```
PODS_BEFORE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry \
  -o jsonpath='{range .items[*]}{.metadata.uid}{"\n"}{end}' | sort | tr '\n' '
  ')"
kubectl --context "$SELF_CONTEXT" apply -f vpa_lab.yaml
VPA_TARGET=""
for attempt in $(seq 1 36); do
  VPA_TARGET="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get vpa devops-
may-cry \
  -o jsonpath='{.status.recommendation.containerRecommendations[0].target.cpu}
{" / "}{.status.recommendation.containerRecommendations[0].target.memory}' \
  2>/dev/null || true)"
  test "$VPA_TARGET" != " / " && test -n "$VPA_TARGET" && break
  printf 'waiting for VPA recommendation (%s/36)\r' "$attempt"
  sleep 10
done
test "$VPA_TARGET" != " / "
printf 'VPA target cpu / memory: %s\n' "$VPA_TARGET"
kubectl --context "$SELF_CONTEXT" -n traffic-lab describe vpa devops-may-cry \
  | sed -n '/Recommendation:/, $p'
PODS_AFTER="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry \
  -o jsonpath='{range .items[*]}{.metadata.uid}{"\n"}{end}' | sort | tr '\n' '
  ')"
printf 'pod UIDs before: %s\npod UIDs after:  %s\n' "$PODS_BEFORE" "$PODS_AFTER"
test "$PODS_BEFORE" = "$PODS_AFTER"
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete vpa devops-may-cry --ignore-not-found
```

✅ После команды

Ожидаю: VPA публикует target CPU/memory; UID двух Pod'ов до и после совпадают, потому что `updateMode: Off`. Объект VPA удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

VPA анализирует usage samples и предлагает requests. Рекомендация — вход для инженерного решения, а не готовый production values-файл. Её сверяют с p95/p99, startup, OOM, throttling и сезонностью.

⚠️ ЛОВУШКА

HPA по CPU utilization делит usage на request. Если VPA одновременно меняет request, горизонтальная формула тоже меняется. Совместную политику проектируют явно: например, VPA рекомендует, а человек обновляет requests; либо HPA масштабирует по метрике, которая не конфликтует с изменяемым request.

ПЕРЕХОД

«Pod'ы мы масштабировать умеем. Остался случай, когда третий Pod корректный, но двум нодам физически некуда его поставить».

Время: 120:00–139:00 плюс создание VM

ЭКРАН: `infra/managed/terraform/kubernetes.tf` →

`03_ЧАСТЬ_3_SCALING/32_3.11_cluster_autoscaler/devops_may_cry.yaml` → Yandex Cloud UI и терминал.

ГОВОРЮ

«Для Cluster Autoscaler переключаюсь на managed-кластер. Self-managed CA тоже возможен, но ему нужен provider integration и права создавать ноды. В managed node group эту интеграцию обслуживает облако.

Продолжаю тот же Terraform state, которым managed-кластер создан в части 1. Второй одноимённый кластер из другого state не создаю. Перед plan обновляю IAM-токен в текущем терминале: с момента первой части могло пройти больше срока его жизни.

В `infra/managed/terraform/kubernetes.tf` один dynamic block выбирает fixed scale, другой — auto scale. Включаю диапазон от двух до пяти нод. Plan должен менять существующую node group, а не собирать новый кластер. После apply проверяю baseline — ровно две Ready-ноды».

ПОКАЗАТЬ

- `enable_autoscaling` в `main.tf`.
- `fixed_scale` и `auto_scale` в `kubernetes.tf`.
- `min=2`, `max=5`, `initial=node_count`.
- В Yandex Cloud UI — node group и autoscaling range после apply.

📄 V2-C3-S12-C01 · Включить Cluster Autoscaler в существующей managed node group

```
MANAGED_TF="$REPO_ROOT/infra/managed/terraform"
cd "$MANAGED_TF" || exit
export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN" && test -n "$YC_CLOUD_ID" && test -n "$YC_FOLDER_ID"
terraform init
terraform validate
terraform plan \
    -var='enable_autoscaling=true' \
    -var='autoscaling_min_nodes=2' \
    -var='autoscaling_max_nodes=5' \
    -out=/tmp/video2-cluster-autoscaler.tfplan \
    >/tmp/video2-cluster-autoscaler-plan.log
terraform show -json /tmp/video2-cluster-autoscaler.tfplan \
    | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] | @tsv'
terraform apply -no-color /tmp/video2-cluster-autoscaler.tfplan \
    >/tmp/video2-cluster-autoscaler-apply.log
grep -E '^(Apply complete|Outputs:|[:alnum:])+ = <sensitive>)' \
    /tmp/video2-cluster-autoscaler-apply.log
yc managed-kubernetes node-group list --format json \
    | jq -r '.[0] | [.name, .status] | @tsv'
kubectl --context "$MANAGED_CONTEXT" get nodes -o wide
BASELINE_NODES="$(kubectl --context "$MANAGED_CONTEXT" get nodes \
    --no-headers | wc -l | tr -d ' ')"
test "$BASELINE_NODES" -eq 2
```

✓ После команды

Ожидаю: IAM-токен обновлён в текущем shell; Terraform меняет существующую node group с fixed scale на auto scale `[2..5]`; перед нагрузкой в managed-кластере ровно две Ready-ноды.

🗣 ГОВОРЮ ПЕРЕД PENDING

«Применяю DEVOPS MAY CRY в managed-кластер. У каждой ноды два vCPU, а request каждой реплики ставлю `[1500m]`. На одну ноду помещается только одна такая реплика. Чтобы rolling update не создал промежуточный surge Pod и не запустил autoscaling раньше нужного кадра, здесь делаю контролируемый lab-reset: временно scale в ноль, меняю request и затем сразу запускаю три реплики. Для production-деплойа так делать не надо; здесь это только способ получить воспроизводимое доказательство scheduler-а.

Две ноды принимают две реплики; третья гарантированно остаётся Pending.

Сначала доказываю причину через событие `[Insufficient cpu]`. Высокая загрузка CPU сама по себе не является сигналом Cluster Autoscaler. Сигнал — unschedulable Pod, который смог бы разместиться на новой ноде подходящей группы.

Затем жду два факта: число Ready-нод стало больше baseline, и тот же Pending Pod перешёл в Running. После доказательства возвращаю request `[100m]` и две реплики».

🖥 ПОКАЗАТЬ

- `[devops_may_cry.yaml]`: две replicas, request `[100m]`, PDB и topology spread.
- Контролируемый lab-reset: scale в ноль → patch `[1500m]` → scale до трёх.
- Events Pending Pod.
- В отдельный момент — Yandex Cloud UI, новая VM в node group.
- Финальный `[get pods -o wide]`: третья реплика сидит на новой ноде.

📑 V2-C3-S12-C02 · Создать 1500m Pending и доказать появление новой ноды

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/32_3.11_cluster_autoscaler"
cd "$LAB_DIR" || exit
kubectl --context "$MANAGED_CONTEXT" apply -f devops_may_cry.yaml
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry --replicas=0
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab patch deploy devops-may-cry \
    --type=json \
    -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/c
pu","value":"1500m"}]'
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry --replicas=3
PENDING_POD=""
for _attempt in $(seq 1 30); do
    PENDING_POD="$(kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Pending \
        -o jsonpath='{.items[0].metadata.name}' 2>/dev/null || true)"
    test -n "$PENDING_POD" && break
    sleep 2
done
test -n "$PENDING_POD"
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab describe pod "$PENDING_POD" \
    | sed -n '/Events:/, $p'
for _attempt in $(seq 1 60); do
    PHASE="$(kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pod
"$PENDING_POD" \
        -o jsonpath='{.status.phase}')"
    CURRENT_NODES="$(kubectl --context "$MANAGED_CONTEXT" get nodes \
        --no-headers | wc -l | tr -d ' ')"
    printf 'nodes=%s pod=%s phase=%s\n' "$CURRENT_NODES" "$PENDING_POD" "$PHASE"
```

```
test "$CURRENT_NODES" -gt "$BASELINE_NODES" && test "$PHASE" = "Running" &&
break
sleep 10
done
test "$CURRENT_NODES" -gt "$BASELINE_NODES"
test "$PHASE" = "Running"
kubectl --context "$MANAGED_CONTEXT" get nodes -o wide
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pods \
-l app=devops-may-cry -o wide
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab patch deploy devops-may-cry \
--type=json \
-
p='[{"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/c
pu","value":"100m"}]'
```

```
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
scale deploy/devops-may-cry --replicas=2
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
rollout status deploy/devops-may-cry --timeout=180s
```

✓ После команды

Ожидая: после контролируемого lab-reset в ноль сразу создаются три реплики с request `1500m`; третья сначала Pending с событием `Insufficient cpu`. Затем число Ready-нод становится больше двух и тот же Pod переходит в Running. В конце request и replicas возвращены к baseline.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Scheduler создаёт доказательство нехватки места — Pending с причиной. Cluster Autoscaler моделирует, поможет ли новая нода, просит provider увеличить node group, а после регистрации kubelet scheduler назначает тот же Pod.

⚠️ PRODUCTION NUANCE

Нода может не появиться из-за quota, недоступной зоны, taint, affinity, слишком большого request или неподходящей node group. Поэтому acceptance criterion включает Pending event, изменение node group, Ready-ноду и Running Pod. Одной строки «autoscaling enabled» в Terraform мало.

ПЕРЕХОД

«Теперь понятно, как Kubernetes измеряет, ограничивает и масштабирует приложение. В четвёртой части пройдем путь запроса: DNS, Service, EndpointSlice, kube-proxy, NodePort и внешний HA-вход».

Время: 139:00–143:00

ЭКРАН: итоговая доска Ч3.

ГОВОРЮ

«Соберу цепочку без новых терминов.

Metrics Server дал текущие resource samples. Requests зарезервировали место у scheduler. Limits превратились в cgroup. Memory limit дал OOMKilled, CPU limit — throttling без restart. Kubelet отдельно показал Evicted. HPA увеличил число Pod'ов, VPA в режиме Off посчитал request, Cluster Autoscaler добавил ноду для доказанного Pending.

Это три разных контура. HPA не создаёт VM. VPA не лечит нехватку capacity. Cluster Autoscaler не исправляет неверный request. Когда каждый отвечает на свой сигнал, масштабирование можно проверять по фактам, а не по одному зелёному статусу».

ЧЕКПОЙНТ ПЕРЕД Ч4

- Self-managed context остался `kubespray`.
- Managed context остался `yc-managed`.
- HPA, VPA и временные fault-workload удалены или возвращены к baseline.
- Managed DEVOPS MAY CRY: две реплики, request `100m`.
- Node group остаётся auto scale `2..5`; её cleanup будет в финальном cleanup выпуска.